THE UNIVERSITY of TULSA
Tandy School of Computer Science

THE UNIVERSITY OF TULSA

Project 4: Neural Networks, CNNs, and Transformers

CS-4103 Fundamentals of Machine Learning

In the Fundamentals of Machine Learning course, your projects require you to write and run Python code. You will implement, modify, or run one or more learning algorithms, train models with multiple datasets, test the accuracy of your models, and produce graphs and tables showing the performance of your models.

You will be given a code shell to start with. It comes as three notebooks: `Assignment 4.1` holds the layered, feedforward network part, `Assignment 4.2` the convolutional network part, and `Assignment 4.3` the attention part. You will build every network from PyTorch layers, and PyTorch computes the gradients, so you do not implement backpropagation. The code shell provides the data loading, the training and testing functions, and the plots. Your task is to build each network architecture, choose how many epochs to train it, run the experiments, and report on the results. Your work is the notebook cells marked `TODO` and the report. You will not need to import any libraries besides those provided. Use the code shell as a guide, but feel free to restructure the code as you prefer. You may discuss algorithms with other students or chatbots, but it is up to you to write the code and the report.

Run the notebooks in Google Colab. Open each one from its link below, then save your own copy with *File* > *Save a copy in Drive* and work in that copy. For `Assignment 4.2`, also choose *Runtime* > *Change runtime type* > *T4 GPU*.

- Assignment 4.1: Layered, Feedforward Neural Networks
- Assignment 4.2: Convolutional Neural Networks
- Assignment 4.3: Attention

The same notebooks are in the course repository if you prefer to run them on your own computer.

Layered, Feedforward Neural Networks

1 Overview

A basic neural network is a stack of dense linear layers with nonlinear activation functions between them. You will build three networks of different depth and width from PyTorch layers, train them with stochastic gradient descent on three classification datasets, and compare how well each one fits and generalizes.

Read the Harvey items *Neural Networks: Perceptron, Multilayer Feedforward Networks* and *Deep Neural Networks & Computation Graphs*.

2 Algorithm

Each network uses two kinds of layers: dense linear layers (`nn.Linear`) and the ReLU activation function (`nn.ReLU`). The last layer is linear, with one output per class. Train every network on the cross-entropy loss with `torch.optim.SGD`, using a learning rate of $\eta = 0.01$, an L2 (ridge) regularization strength of $\lambda = 0.001$ (the `weight_decay` argument), and minibatches of 32 training instances. The code shell provides the training function.

3 Datasets

You have used the first two datasets before.

Handwritten Digits This is the same UCI ML handwritten digits dataset that you used on the previous projects.

Statlog (German Credit Data) This UCI dataset includes data about 1000 loan applications with 20 features each. The goal is to predict whether or not an applicant is creditworthy. The data has been formatted as a numeric dataset.

Mushroom This UCI dataset from the Audubon Society Field Guide includes data about 8124 mushrooms, with 22 categorical features such as cap shape and odor. Each mushroom is either poisonous (1) or edible (0). Veil type has a single value and has been dropped. The other 21 features have been one-hot encoded into 116 columns of 0s and 1s, with missing values kept as their own category.

4 Analysis

Every model is evaluated with 5-fold cross-validation. The code shell splits each dataset into five folds that keep its class proportions. Each model is trained five times, each time on four folds, and tested on the fold it did not see; the features are standardized with the statistics of the four training folds. The results are averaged over the five runs, which makes them much less dependent on one lucky or unlucky split than a single train and test split. For each dataset, you will train three models.

`Model 1:` A small network with one hidden layer of 16 units and an output layer.

1. Linear(input count, 16) + ReLU
2. Linear(16, class count)

`Model 2:` A deeper network with four hidden layers of 100 units, each followed by a ReLU, and an output layer.

1. Linear(input count, 100) + ReLU
2. Linear(100, 100) + ReLU
3. Linear(100, 100) + ReLU
4. Linear(100, 100) + ReLU
5. Linear(100, class count)

`Model 3:` A wide network with one hidden layer, like `Model 1`, whose width L is chosen so that the network has about the same number of parameters as `Model 2`. A linear layer from a inputs to b outputs has $(a+1)b$ parameters, so with n inputs and k classes Model 3 has $(n+1)L+(L+1)k$. Setting that equal to the parameter count of Model 2 and solving for L gives $L = 504$ for Digits, $L = 1417$ for German Credit, and $L = 355$ for Mushroom. The code shell sets these values.

1. Linear(input count, L) + ReLU
2. Linear(L, class count)

Train each model to convergence, and choose the number of epochs for each dataset from the loss plots. The code shell plots the training and testing loss of every model for each epoch, averaged over the five folds.

5 Report & Scoring

Give the parameter counts of all three models on each dataset and the number of epochs you chose for each dataset. Include each loss plot, and each model's mean testing accuracy over the five folds with its standard deviation. Compare the deep network (Model 2) with the wide network (Model 3), say whether the differences between models are larger than the spread across folds, and note where a model's test loss rises while its training loss keeps falling.

- **5pts** - The parameter counts of the three models on each dataset, and the number of epochs you chose for each dataset and why.
- **45pts** - The loss plot, testing accuracy, and analysis of each of the nine models are worth 5 points.

Convolutional Neural Networks

1 Overview

A convolutional network learns small filters that slide across an image, so the same pattern is detected anywhere in the image. You will build five convolutional networks in PyTorch for image classification and compare the effect of the number of filters, dropout, batch normalization, and depth.

Read the Harvey item *Convolutional Neural Networks*, and the *Computation Graphs & Deep Neural Networks* slides, slides 20 to 31.

2 Datasets

Fashion-MNIST Grayscale photos of clothing in ten classes, such as shirt, sneaker, and bag.

MNIST Digits Handwritten digits 0 to 9.

Each image is 28 by 28 pixels. Both datasets come split into 60,000 training images and 10,000 testing images. The code shell downloads them, joins the two splits into one set of 70,000 images per dataset, and keeps a fixed sample of 20,000 of them, 2,000 from each class. Every model is evaluated with 5-fold cross-validation on that sample: the 20,000 images are split into five folds of 4,000, and each model is trained five times, each time on four folds (16,000 images), and tested on the fold it did not see.

3 Analysis

Every convolution uses 3×3 filters with `padding="same"`, and every max pooling layer uses a 2×2 pool. Every model ends with a linear layer of 10 outputs. Train each model on the cross-entropy loss with the Adam optimizer.

Model 1A: Two blocks, each of two convolutions with ReLUs followed by max pooling, then a linear layer of 1024 units with a ReLU, then the output layer. The first block's convolutions have 16 filters and the second block's have 32.

Model 1B: Model 1A with 64 filters in the first block and 128 in the second.

Model 2: Model 1B with a dropout layer of rate 0.5 after each max pooling layer.

Model 3: Model 1B with a batch normalization layer after the ReLU of each convolution.

Model 4: Three blocks of three convolutions with 64, 128, and 256 filters. Each convolution is followed by a ReLU and batch normalization, and each block ends with max pooling and dropout of rate 0.5. Then linear layers of 1024 and 512 units with ReLUs, then the output layer.

Train every model with a learning rate of 0.001. Start each model with more epochs than it needs, and from its loss plot, set the number of epochs to the point of diminishing returns, where more training stops lowering the testing loss. Each model uses the same number of epochs on both datasets. The code shell also trains Model 4 a second time with a learning rate of 0.01.

4 Report & Scoring

Include each model's loss and accuracy plots, the epochs you chose, its training time per fold, and its mean testing accuracy over the five folds with its standard deviation. Compare Models 1A and 1B in accuracy and training time, Model 2 and Model 3 against Model 1B, and Model 4 against the rest. For Model 4, also describe what changed at a learning rate of 0.01, in the loss curves and in the accuracy, and why.

- **30pts** - Results and analysis of each model on each dataset, with Model 4 covering both learning rates, are worth 3 points.

Transformers

1 Overview

Attention lets every token in a sequence weigh every other token when it computes its new representation, and it is the core of the transformer architecture. You will write scaled dot-product attention, run it inside a provided multi-head attention layer, then use the weights of BERT, a pretrained transformer, to see what trained attention heads look at.

The task is masked word prediction, the task BERT was trained on. BERT was pretrained on about 3.3 billion words of English text, BookCorpus and English Wikipedia: about 15% of the tokens (words and pieces of words) in each training sentence were hidden behind a `[MASK]` token, and BERT learned to predict each hidden word from the words on both sides of it. You will not train anything and need no dataset. You will run the pretrained BERT on two sentences, watch it fill in a hidden word, and look at the attention it learned while doing that task.

Read the Harvey item *Transformers Summary (Joe Shymanski)*, slide 5, and the paper *Attention Is All You Need* in *Deep Sequence Modeling (RNNs, LSTMs, Transformers)*.

2 Algorithm

Scaled dot-product attention computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where d_k is the size of each head. The code shell provides a multi-head attention layer that calls your function. It projects the input X into queries, keys, and values for every head, applies attention in each head, and mixes the heads' outputs with an output projection W^O:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \qquad \text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V).$$

3 Analysis

- Fill in the code for scaled dot-product attention.
- Use two sentences. The first is given: "The trophy did not fit in the suitcase because it was too big." For the second, write a sentence of your own, 8 to 20 words long, and choose one of its words for BERT to fill in.
- The code shell hides the word "big" in the given sentence and your chosen word in yours, and prints BERT's five most likely words for each.
- Run your layer with random weights on both sentences and plot the attention matrix of each of its 12 heads.
- Load the weights of BERT's first layer (layer 0) into your layer, run it on both sentences, and plot its heads. The code shell prints the largest difference from the attention BERT computes itself.
- Choose a later BERT layer, from 1 to 11, and repeat.

4 Report & Scoring

Include BERT's guesses for the hidden words, the heatmaps of all three runs on both sentences, and the printed differences. Say whether BERT's guesses were sensible and which words give the clues. Describe the patterns of at least three heads in layer 0 and in your later layer, and compare them with each other and with the untrained heads.

- **8pts** - Your scaled dot-product attention: with it, the provided layer reproduces BERT's attention in layer 0 and in your later layer, on both sentences.
- **3pts** - BERT's guesses for the hidden words in both sentences, whether they were sensible, and which words give the clues.
- **9pts** - The heatmaps and analysis of the untrained layer, BERT layer 0, and your later layer are worth 3 points each.

Submission

Notebooks: Submit the three completed notebooks with every cell run and its output visible. In Colab, download each one with *File* > *Download* > *Download .ipynb*. Cleared notebooks will be graded as though the code did not run.

Report: Submit the report as a single PDF containing the plots, tables, and comments described above.

Where: Submit both through Harvey.